CAHIER DES CHARGES TECHNIQUE

Application Mobile Touristique Multi-Publics — Architecture Modèle-Vue-Contrôleur (MVC)

1. Contexte du Projet

Le projet consiste à concevoir et développer une application mobile de tourisme innovante. Elle a pour ambition de dynamiser l'attractivité touristique en ciblant simultanément deux types d'utilisateurs aux comportements et besoins radicalement distincts :

- **Les touristes étrangers :** Nécessitent une assistance à l'orientation, des interfaces multilingues, des modules de conversion de devises, ainsi qu'un accès aux données critiques en mode déconnecté (hors-ligne) pour pallier l'absence de réseau cellulaire à leur arrivée.
- **Les résidents locaux :** Recherchent des micro-aventures, des événements éphémères (festivals, marchés, expositions) et des bons plans de proximité pour occuper leurs week-ends et temps libres.

Pour garantir un déploiement rapide et efficace sur les plateformes iOS et Android à partir d'un code source unique, l'environnement de développement retenu est **Android Studio** associé au framework multiplateforme de Google, propulsé par le langage **Dart**.

2. Spécification des Besoins

2.1. Besoins Fonctionnels (Ce que l'application fait)

- **Orientation Automatique du Profil :** Détection ou sélection initiale permettant de basculer l'affichage en mode "Étranger" ou "Local".
- **Moteur de Recherche & Filtrage :** Recherche multicritère des sites touristiques et des activités par pertinence, catégorie (plages, monuments, restaurants) et distance géographique.
- **Cartographie & Itinéraires :** Visualisation des points d'intérêt sur une carte interactive avec calcul d'itinéraires en temps réel.
- **Gestion des Événements (Spécial Local) :** Agenda dynamique permettant de suivre et de planifier les sorties locales de la semaine.
- **Système d'Avis et d'Évaluation :** Possibilité pour la communauté de noter les sites et de laisser des commentaires constructifs.

2.2. Besoins Non Fonctionnels (Comment l'application le fait)

- **Performance & Fluidité :** Temps de réponse des affichages d'interfaces inférieur à 2 secondes.
- **Disponibilité Hors-Ligne (Mode Cache) :** Consultation des fiches de sites touristiques et des cartes sauvegardées préalablement sans connexion internet active.
- **Sécurité des Données :** Chiffrement obligatoire des mots de passe utilisateurs (hachage côté serveur) et sécurisation des requêtes HTTP via des jetons d'accès éphémères.
- **Ergonomie & Accessibilité :** Interface moderne, épurée, respectant les standards de design Material Design et s'adaptant à toutes les tailles d'écrans mobiles.

3. Architecture Technique Alignée sur le Modèle MVC

Le projet adopte une séparation stricte des responsabilités selon le patron d'architecture **MVC (Modèle-Vue-Contrôleur)**. Cette méthodologie garantit la maintenabilité du code, la sécurité des processus et l'indépendance de l'interface visuelle vis-à-vis du stockage des données.

> **Règle d'or de l'architecture :** La Présentation (Vue) ne communique jamais directement avec la Base de Données (Modèle). Toutes les requêtes transitent obligatoirement par la Logique Métier (Contrôleur).

3.1. Partie I : La Présentation (La Fontaine / "Vue")

La Présentation regroupe l'ensemble des éléments visibles et manipulables par l'utilisateur final. Développée sur Android Studio, elle traduit graphiquement les réponses fournies par le serveur.

- **Composant Technologique :** `Flutter` / `Dart`
- **Rôle applicatif :** Capturer les interactions de l'utilisateur (clics, gestes, saisies), envoyer des requêtes asynchrones au serveur via des appels HTTP sécurisés (REST/JSON) et mettre à jour dynamiquement l'affichage visuel.
- **Gestion d'état locale :** Utilisation d'un gestionnaire (ex: Bloc ou Provider) pour mémoriser temporairement le profil de l'utilisateur (Étranger ou Local) et ses préférences de navigation.
- **Cache local :** Utilisation d'une base de données embarquée légère (ex: Hive ou SQLite local) pour le stockage des données indispensables au mode hors-ligne.

3.2. Partie II : La Logique Métier (Le Baptême / "Contrôleur")

Le "Baptême" est l'intelligence centrale de l'application installée sur un serveur distant. Il joue le rôle de pivot incontournable entre les demandes de l'utilisateur et le stockage des données.

- **Composant Technologique :** `Node.js (Express)` ou `Firebase Cloud Functions`
- **Point d'entrée (API REST) :** Exposition d'URLs sécurisées permettant à la Fontaine de dialoguer avec le serveur (Exemples : `GET /api/sites`, `POST /api/avis`).
- **Règles Métiers :** Calcul des distances géographiques, filtrage des événements selon la date, validation de la structure des données reçues.
- **Sécurité & Authentification :** Vérification de l'identité des utilisateurs grâce à des jetons sécurisés JWT (JSON Web Tokens). Le Contrôleur filtre les droits d'accès (un touriste lit les informations, seul l'administrateur peut modifier un site).

3.3. Partie III : La Base de Données (Le Stockage / "Modèle")

La Base de Données assure la persistance et l'intégrité de toutes les informations de la plateforme. Elle stocke les données brutes manipulées par le Contrôleur.

- **Composant Technologique :** `PostgreSQL / MySQL` (Relationnel) ou `MongoDB / Firestore` (NoSQL)
- **Rôle applicatif :** Enregistrer de manière pérenne et structurée les tables et les collections essentielles au fonctionnement de l'écosystème.

TABLE / ENTITÉ	ATTRIBUTS PRINCIPAUX	RÔLE AU SEIN DU SYSTÈME
Utilisateurs	ID, Nom, Email, Mot de passe (haché), Type (Étranger / Local), Langue	Identification des profils et mémorisation des préférences d'affichage.
Sites Touristiques	ID, Nom, Description, Latitude, Longitude, Catégorie, Url_Photos	Répertoire des points d'intérêt cartographiques et géolocalisés.
Avis & Événements	ID, Id_Site, Id_Utilisateur, Note, Commentaire, Date_Evenement	Gestion des interactions sociales et de l'agenda culturel dynamique.